Laboratory Work 09

Spring Transaction Management
and Database Schema Versioning

Arzum Kassymzhanov | IT1-2404IS
3 November 2026

Aim and outcome

I implemented atomic account transfers, transaction propagation experiments, independent database reads at two isolation levels, optimistic locking and versioned database migrations. The application and 18 integration tests confirm rollback behavior and prevent a stale account update from overwriting a committed change.

Environment	Verified configuration
Application	Standalone lab09 module in spring-lab-01
Framework	Spring Boot 3.5.16 with Spring Data JPA
Runtime	Java 25.0.2; compiled for Java 17; Maven 3.9.16
Database	H2 2.3.232; memory database and file database restart proof
Schema tool	Flyway; Hibernate ddl-auto=validate
Local demo	http://127.0.0.1:8089; loopback binding

Main observations

Experiment	Observed result
Transfer and rollback	1000/500 becomes 900/600; failures restore 1000/500
Propagation	REQUIRED leaves 0 audit rows; REQUIRES_NEW leaves 1
Isolation on H2	READ_COMMITTED sees 1050; REPEATABLE_READ sees 1000
Optimistic locking	Stale commit rejected; balance 1010; version 1
Migration lifecycle	2 migrations, then 0 on restart, then 2 new versions

Evidence is included as machine-readable API responses, complete Maven/Surefire output and application startup logs. Report figures reproduce excerpts from those saved observations.

Task A Declarative transactions and rollback

The service method is the business transaction boundary: loading both accounts, checking the amount, debiting and crediting all take place in one transaction. Account balances use BigDecimal and DECIMAL(19,2). The database also rejects negative balances.

```
@Transactional
public void transfer(Long fromId, Long toId, BigDecimal amount) {
    Account from = get(fromId);
    Account to = get(toId);
    from.debit(amount);
    to.credit(amount);
    repository.save(from);
    repository.save(to);
}
```

Actual API observations | Transactions and rollback

Scenario	Alice	Bob	Result
Initial	1000.00	500.00	total 1500.00
Transfer 100.00	900.00	600.00	committed
Insufficient funds	1000.00	500.00	rolled back
Unchecked error after flush	1000.00	500.00	rolled back
Checked exception, default	900.00	500.00	debit committed
Checked exception, rollbackFor	1000.00	500.00	rolled back

Source: docs/evidence/live-observations.json; POST /api/lab09/experiments

Figure 1. Rendered excerpt of the actual live API response for Task A.

After a transfer of 100.00, Alice has 900.00 and Bob 600.00. Their total remains 1500.00. A transfer of 1001.00 fails with Insufficient funds; neither balance changes.

The unchecked failure experiment calls repository.flush() after the debit and throws before any credit. The balance returns to 1000.00, demonstrating rollback of an SQL update that already reached the database.

Checked exceptions and rollback rules

With the default Spring rollback rules, RuntimeException and Error trigger rollback; a checked exception normally does not. checkedDefault therefore commits the deliberately isolated debit, leaving Alice at 900.00. checkedRollback uses @Transactional(rollbackFor = CheckedTransferException.class), restoring 1000.00. These methods are controlled experiments, not transfer endpoints.

Transaction interception requires a call through a Spring proxy. The controller and experiment coordinator call the injected service bean; a same-object call to another annotated method would not create the intended proxy boundary.

Tasks B and C Propagation and isolation

Propagation with an independent audit bean

The outer transaction debits 10.00, invokes AuditService, and throws an unchecked exception after the inner method returns. REQUIRED joins the outer transaction, so its audit insert also rolls back. REQUIRES_NEW suspends the outer transaction and commits its own audit insert; the audit record remains even though the outer debit rolls back.

Actual API observations | Propagation and H2 isolation

```
REQUIRED       after outer rollback: audit rows = 0, Alice = 1000.00
REQUIRES_NEW   after outer rollback: audit rows = 1, Alice = 1000.00

Isolation          First read   Writer committed   Second read
READ_COMMITTED     1000.00      1050.00            1050.00
REPEATABLE_READ    1000.00      1050.00            1000.00
H2 2.3.232; two independent JDBC connections; direct SQL reads
```

Source: docs/evidence/live-observations.json; each isolation scenario starts at 1000.00

Figure 2. Measured propagation results and two-connection H2 isolation experiment.

The audit methods live in a separate Spring bean. REQUIRES_NEW uses an additional connection while the outer transaction is suspended. Audit entries describe the event independently; their existence must not be interpreted as proof that the outer transfer committed.

Isolation experiment procedure

Reader and writer use two separate JDBC connections with auto-commit disabled. The reader selects Alice at 1000.00; the writer adds 50.00, increments the version, and commits. The reader repeats the SELECT before ending its own transaction. Direct SQL avoids the JPA first-level cache.

On H2 2.3.232, READ_COMMITTED returns 1050.00 on the second read, demonstrating a non-repeatable read. REPEATABLE_READ returns 1000.00 while a fresh committed read returns 1050.00. Both modes prevent dirty reads. REPEATABLE_READ prevents non-repeatable reads of the same row, but phantom handling and other details depend on the database. This experiment tests repeated reads of one row; it does not test predicate phantoms, write skew or serializability.

Other propagation modes

SUPPORTS joins a transaction if present; MANDATORY requires one; NOT_SUPPORTED suspends it; NEVER rejects an active transaction. NESTED uses a savepoint where the transaction manager supports it; it is not a portable substitute for REQUIRES_NEW with JPA.

Task D Optimistic locking

Account maps its version column with @Version. I loaded the same account in two independent EntityManagers with simultaneously active transactions. Both copies had version 0. The first context credited 10.00 and committed. The second tried to credit 20.00 using its stale version.

```
@Version
@Column(nullable = false)
private Long version;

-- Hibernate update includes the expected version:
UPDATE accounts SET balance = ?, version = ?
WHERE id = ? AND version = ?;
```

Actual API observations | Stale update rejected

```
Two independent persistence contexts loaded version 0
Context 1: credit 10.00; commit succeeds
Context 2: credit 20.00; commit fails
Exception: RollbackException caused by optimistic-lock conflict
Final balance: 1010.00; committed version: 1
staleUpdateRejected: true
```

Source: docs/evidence/live-observations.json; ConcurrencyExperiments.optimisticLock

Figure 3. Actual stale-commit result from two independent persistence contexts.

The first commit changed Alice to 1010.00 and version 1. The stale commit failed with RollbackException containing an optimistic-lock cause. Its WHERE version=0 matched no row, so it could not silently replace the committed value with 1020.00. The test verifies that the final database state remains 1010.00 and version 1.

A safe response to a conflict

The failed transaction must be rolled back and its persistence context discarded or refreshed before another attempt. A user-facing service should report the conflict and offer current data. An automatic retry is appropriate only when the complete operation is safe to repeat and its business conditions are rechecked; a money transfer also needs protection against duplicate execution. Reusing the stale entity or blindly replaying a debit can lose updates or perform an operation twice.

What the version field protects

The version check detects competing updates to one versioned entity. It complements transaction atomicity; it does not by itself provide serializable execution across arbitrary multi-row business rules. The isolation experiment increments version explicitly because it performs direct SQL rather than an entity update.

Task E Database schema versioning

Version	Migration	Purpose
V1	V1__create_accounts.sql	Accounts, balances, version and seed data
V2	V2__add_status.sql	ACTIVE status with a non-null default
V3	V3__index_owner_name.sql	Index on owner_name
V4	V4__create_audit_records.sql	Independent audit table

The automated migration experiment queries the quoted flyway_schema_history table and records each version, script, success flag and checksum. It also verifies that IDX_ACCOUNTS_OWNER_NAME exists. A fresh Flyway instance reuses the same schema history and executes zero already-applied versions.

Actual startup logs | Three complete application launches

```
first-v2.log
  Migrating schema "PUBLIC" to version "1 - create accounts"
  Migrating schema "PUBLIC" to version "2 - add status"
  Successfully applied 2 migrations to schema "PUBLIC", now at version v2
restart-v2.log
  Schema "PUBLIC" is up to date. No migration necessary.
upgrade-v4.log
  Migrating schema "PUBLIC" to version "3 - index owner name"
  Migrating schema "PUBLIC" to version "4 - create audit records"
  Successfully applied 2 migrations to schema "PUBLIC", now at version v4
```

Source: docs/evidence/first-v2.log, restart-v2.log, upgrade-v4.log

Figure 4. Excerpts from real application startup logs against the same H2 file.

The separate PowerShell check launches the complete application three times against one persistent H2 file. First, target V2 applies V1 and V2 to an empty database. A full stop and restart at target V2 applies nothing. The final startup at target V4 applies the new index and audit migrations. The first two stages use ddl-auto=none because the audit table is intentionally not present yet; the final stage validates the complete schema.

Applied versioned migrations must remain immutable. Editing an old file changes its checksum and can make validation fail; it also makes different environments disagree about what the same version means. A correction should be a new versioned migration committed alongside the code.

Verification and reproducibility

Actual Maven and Surefire output

```
Test set: kz.iitu.txlab.TransactionLabTests
Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
Spring Boot 3.5.16 | H2 2.3.232 | Java 25.0.2
```

Source: docs/evidence/surefire-summary.txt and maven-verify.log

Figure 5. Actual successful Maven verification and Surefire summary.

Coverage	Verified behavior
Transactions	Successful transfer, conserved total, insufficient funds, flushed rollback
Rollback rules	Checked default commit and explicit rollbackFor
Propagation	REQUIRED audit rollback and REQUIRES_NEW independent commit
Concurrency	READ_COMMITTED, REPEATABLE_READ and stale entity rejection
Migrations	Initial application, no repeat, new versions and index existence
Boundaries and HTTP	Same account, missing destination, fractional cents, blocked destination, 400 and 409

Run from the repository

```
Set-Location 'C:\Labworks\Spring\spring-lab-01-main\lab09'
.\mvnw.cmd clean verify
java -jar .\target\transaction-lab09-1.0.0.jar
# Open http://127.0.0.1:8089
# Stop the application before the full restart check:
.\scripts\verify-migrations.ps1
```

The default database is in memory and starts with Alice 1000.00 and Bob 500.00. The experiment button deliberately resets those sample accounts before each scenario. File persistence is selected with --spring.datasource.url=jdbc:h2:file:./data/txlab. The teaching API binds to 127.0.0.1 and is intended for local, disposable lab data.

Delivered implementation

The lab09 module contains the entity and repositories, transfer and audit services, propagation and concurrency experiments, Flyway migrations, integration tests, a local browser demonstration, HTTP requests, the restart script, saved evidence and this report. The original book application remains a separate project at the repository root.

Answers and conclusions

1 What does Transactional do and where do boundaries belong

@Transactional makes a Spring proxy open or join a transaction and apply commit or rollback rules around a method invocation. It configures propagation, isolation, timeout and read-only behavior. Boundaries normally surround one complete business operation in a service, such as moving money between accounts, rather than each repository call independently.

2 REQUIRED and REQUIRES_NEW

REQUIRED joins an existing physical transaction or starts one if none exists. REQUIRES_NEW always creates an independent transaction and suspends the outer one. In this experiment the shared audit rolled back; the independent audit committed and survived the outer failure.

3 Anomalies and database differences

READ_COMMITTED prevents dirty reads but can allow non-repeatable reads and phantoms. REPEATABLE_READ also prevents changes to previously read row values; phantom behavior depends on the engine and implementation. H2 preserved 1000.00 at REPEATABLE_READ while READ_COMMITTED returned the writer's committed 1050.00. These measurements must not be generalized to every database.

4 How Version detects concurrent updates

Hibernate includes the loaded version in an UPDATE predicate and increments the version on success. A competing commit changes the stored version. The stale update then affects zero rows and fails instead of overwriting the first update. The application must handle the conflict in a new valid transaction.

5 Why migrations are version controlled and immutable

Version control makes schema changes reviewable and deployable with the application. A history table tracks order, checksums and successful execution. Immutability keeps environments consistent and allows validation to detect drift. Fixes are introduced through new migrations rather than edits to deployed files.

6 Choosing Flyway or Liquibase

I chose Flyway because the exercise uses a small ordered sequence of SQL migrations that is easy to review. Liquibase is useful when a team needs structured changesets, preconditions, contexts and labels, or explicit rollback definitions. Either tool still requires database-specific validation and deliberate rollback planning; they should not both own this schema.

Conclusion

I observed atomic transfers and rollback after a flushed SQL update, an independent audit commit with REQUIRES_NEW, database-specific repeatable-read behavior, and rejection of a stale entity update. Flyway applied V1–V2 once, skipped them after a real application restart, and then applied V3–V4. Correct transaction boundaries, explicit conflict handling and immutable migrations jointly protect application data.

References

Spring Framework: docs.spring.io/spring-framework/reference/data-access/transaction/declarative/annotations.html

Spring propagation: docs.spring.io/spring-framework/reference/data-access/transaction/declarative/tx-propagation.html

H2 isolation: h2database.com/html/advanced.html#transaction_isolation